

Foundation and Building Blocks

DENG
Forooz Shahbazi

Information Technology

Week 2 — Learning Goals

From understanding the source to making sound storage decisions

1

Evaluate the source

Ask the right questions about data generation, quality, structure, schema changes and ingestion constraints.

2

Reason about schema

Distinguish fixed vs. flexible schemas and explain how schema choices affect downstream pipelines.

3

Choose storage by workload

Match operational, raw/historical, analytical and streaming workloads to different storage requirements.

4

Evaluate storage reliability

Reason about performance/SLA, scalability, schema evolution, metadata, discovery and lineage.

By the end:

Given a dataset such as NYC Taxi, you can justify source and storage design decisions before choosing a tool.

Where does NYC Taxi data actually come from?

- A taxi finishes a trip.
We eventually see a row containing pickup time, drop-off time, locations, passenger count, fare, payment type...

Where did those values originate?

Generation: Source Systems

- **Definition:**

A **source system** is where data originates — such as:

- 📱 Applications & transactional databases
- 🌐 Message queues
- 📡 IoT devices & sensor networks

- **Key Responsibilities of a Data Engineer:**

- Understand **data generation patterns**: structure, velocity, frequency, and variety
- **Consume**, not control, source systems
- Maintain **communication with system owners** to prevent pipeline breakages

- **Common Types:**

- 📖 **Traditional**: Application + RDBMS (e.g., microservices with dedicated DBs)
- 🌍 **Modern**: IoT swarms sending real-time data to central hubs

Generation: Source Systems

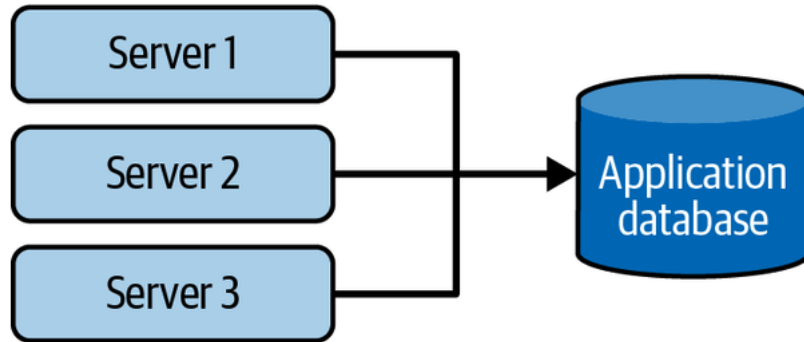


Figure 2-3. Source system example: an application database

1. Application Database (left)

- **Structure:**
 - Multiple servers (applications) write data into a central database.
- **Data type:**
 - Structured, transactional data (orders, payments, customer info, etc.).
- **Access pattern:**
 - Data is stored in tables and queried later (SQL).
- **Use case:**
 - Best for systems of record, transactional consistency, and historical queries.
- **Flow:**
 - Writes happen first, then consumers (analytics, reports, ETL jobs) read later.

👉 **Analogy:** Like a library where information is neatly cataloged and you look it up when needed.

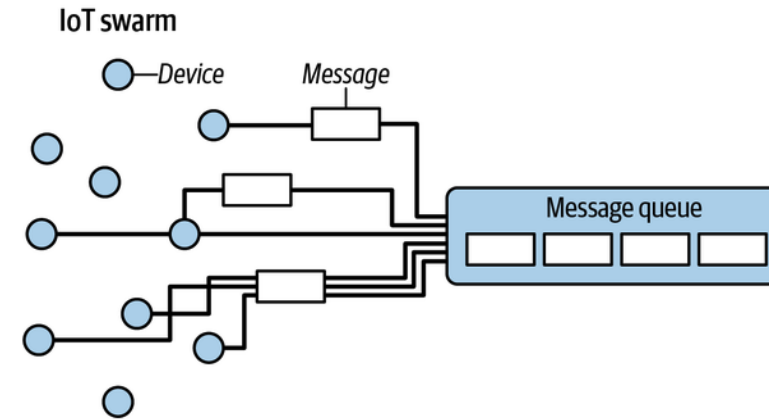


Figure 2-4. Source system example: an IoT swarm and message queue

2. Message Queue (right, with IoT swarm)

- **Structure:**
 - Many producers (IoT devices, apps, services) continuously push messages into a queue.
- **Data type:**
 - Event streams, logs, sensor data, notifications. Often semi-structured (JSON, Avro, etc.).
- **Access pattern:**
 - Consumers "subscribe" and read messages in order, often in near real time.
- **Use case:**
 - Best for streaming data, decoupling producers and consumers, handling bursts of events.
- **Flow:**
 - Messages are ephemeral — they may expire or be consumed once. Not stored permanently like a database.

👉 **Analogy:** Like a conveyor belt in a factory — items flow by, and workers (consumers) pick them up to process.



You are the **data engineer**. What do you need to know?

- The NYC Taxi team tells you:
**“Here is our trip data.
Please ingest it into our analytics platform.”**
- Before building anything, **what questions would you ask them?**



Evaluating Source Systems: What Data Engineers Must Consider

Key Evaluation Areas and Questions you need to ask yourself:

Source Type & Data Generation

- What is the source? App, IoT swarm, etc.?
- How often & how much data is generated?
- Is data persisted long term or ephemeral?

Data Quality & Consistency

- Frequency of nulls, bad formats, duplicates, or late-arriving data
- Are data-quality checks in place?

Schema & Structure

- Fixed vs. Schemaless models: *Schemaless* doesn't mean the absence of schema. Rather, it means that the application defines the schema as data is written.
- How are schema changes managed & communicated?
- Do joins span multiple systems or tables?

Ingestion & State Handling

- Snapshot (a complete copy of the database's state) vs. CDC (change data capture: tracks and captures a stream of **change events**)?
- Pull frequency and load impact on source systems?

Schemaless vs. Fixed Schema

- **Fixed schema (traditional databases)**

- The **schema is predefined** in the database itself.
- Example: In a relational database (like PostgreSQL or MySQL), you define a table first:

```
CREATE TABLE orders (  
  order_id INT,  
  customer_id INT,  
  amount DECIMAL(10,2),  
  order_date DATE  
);
```

The database **enforces this schema**: every row must have those columns, with those data types.

Schemaless (NoSQL, JSON stores, message queues, etc.)
The database itself does **not require a fixed schema upfront**.

Instead, the **application decides** what fields to include when it writes a record.

Example: In MongoDB (a schemaless database), you can insert documents like:

```
{ "order_id": 1, "customer_id": 123, "amount": 45.50 }  
{ "order_id": 2, "customer_id": 456, "amount": 99.99,  
  "discount_code": "SUMMER" }
```

- Notice: the first record has no discount_code, the second one does.
- MongoDB doesn't complain — because it doesn't enforce a fixed schema.

Schema-on-Write vs. Schema-on-Read

Example & Discussion



Schema-on-Write (Structured)

Data must conform to a defined schema before it is written.

Example – NYC Taxi (CSV with defined schema)

trip_id	tpep_pickup_datetime	pickup_longitude	...	fare_amount
STRING	TIMESTAMP	DOUBLE	...	DOUBLE

- ✓ New data is validated against the schema
- ✓ Inconsistent or unexpected fields are rejected
- ✓ Downstream processing is more predictable
- ✓ Changes require coordination (schema evolution)

VS



Schema-on-Read (Schemaless)

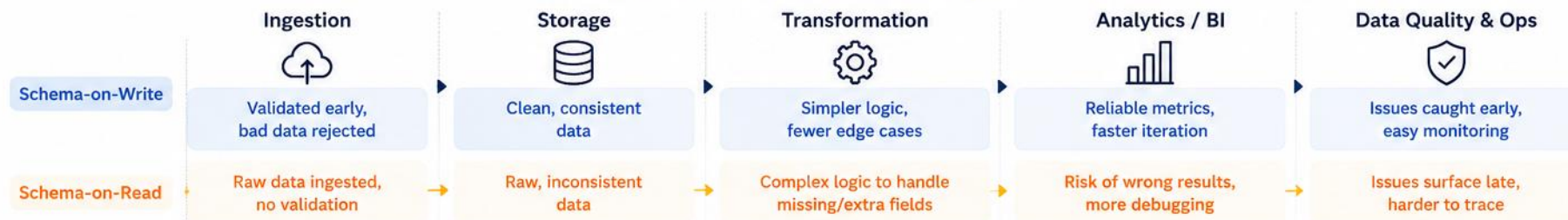
Raw data is written as-is; schema is applied later when reading.

Example – NYC Taxi (Raw JSON as stored)

```
{
  "trip_id": "id_123",
  "tpep_pickup_datetime": "2024-01-01T08:15:00",
  "pickup_longitude": -73.9857,
  "fare_amount": 23.45,
  "payment_type": "credit",           // new field appears
  "surge_multiplier": 1.5           // sometimes present
}
```

- ✓ Any new or missing fields are accepted
- ✓ Interpretation happens during queries/processing
- ✓ Flexibility now, complexity (and surprises) later

Effect on the Downstream Pipeline



Schemaless can be powerful for agility, but it pushes complexity downstream. Data engineers must balance flexibility with reliability.



Is NY Taxi Schemaless or not?



How Do I Know If My Data Source Is Schemaless?

The key question: Does the system enforce a predefined structure when the data is written?



YES → Schema-Enforced (Schema-on-Write)

Data must conform to the schema before it is written.



NO → Flexible / Schemaless (Schema-on-Read)

Data can be written with varying structure; schema applied later.



How to Check (NYC Taxi Data)

- 1 Read the data documentation**
Does it specify a fixed schema and data types? (e.g., data dictionary for NYC TLC trip record)
- 2 Inspect the data samples**
Are all files consistent in columns and types?
Are there missing or extra fields?
Do data types vary (e.g., number vs. text)?
- 3 Test with unexpected data**
What happens if a value has the wrong type (e.g., "FREE" in fare_amount)?
 - If the source **rejects** it → schema-enforced
 - If it **accepts** it → flexible/schemaless
- 4 Ask the source system owners**
Is there a schema validation at the source (e.g., database constraints, schema registry)?



Example: NYC Taxi Data

- ✓ CSV has a **STRUCTURE** (columns) but not strict enforcement

NYC Taxi CSV (example)

VendorID	pickup_datetime	fare_amount
1	2026-01-01 10:30:00	24.50

- CSV defines columns, so there is a structure. But CSV itself **does not enforce** data types.



Tomorrow you could receive:

VendorID	pickup_datetime	fare_amount
1	2026-01-01 10:30:00	FREE

- ⚠ The file still exists perfectly. The **problem appears** when your pipeline tries to interpret "FREE" as a number.



Two Questions to Ask

- 1 Does the data have a schema/structure?**
 - e.g., columns in CSV, fields in JSON
- 2 Who enforces that schema, and when?**
 - At the source when writing?
 - Later when reading/processing?



This second question is what really matters for data engineering.

Common Sources: Where Is the Schema Enforced?



PostgreSQL / MySQL

Table schema & types defined first

✓ Enforced



CSV Files

Structure exists (columns), but not enforced

⚠ Not enforced



JSON Files

Records may have different fields

⚠ Not enforced



MongoDB

Flexible by default (unless validation configured)

⚠ Not enforced



Kafka (plain JSON)

Kafka does not enforce JSON schema

⚠ Not enforced



Kafka + Schema Registry

Messages validated against Avro/Protobuf/JSON schema

✓ Enforced



Parquet Files

File contains explicit schema and typed columns

✓ Has schema



Takeaway

NYC Taxi CSV has a structure (columns), but the source **does not strictly enforce** types or values. It is therefore a **flexible/schemeless source**, and the schema is applied (and validated) later in your pipeline.

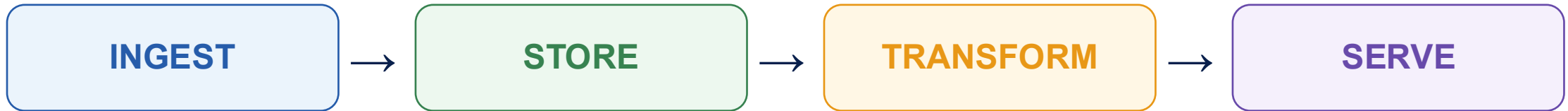


As data engineers, our job is to understand the source deeply before we build anything. Clarity on schema and enforcement is the foundation of a reliable pipeline.



Storage in Data Engineering

The question is not “Where can I put the data?” — it is “What does the workload need?”



Storage affects every stage around it

How quickly can we ingest? • How efficiently can we transform? • How fast can consumers query?

Key idea There is no single “best” storage system. The right choice depends on the workload, access pattern, scale, latency, durability and cost.

Modern cloud platforms may also separate storage from compute, allowing each to scale independently.

Different Data, Different Storage Needs

One architecture may use several storage systems because the workloads are different.

Operational data

Fast reads & writes

Current application
state

e.g. relational / NoSQL
DB

Raw / historical data

Cheap + durable

Large files, long
retention

e.g. object storage

Analytical data

Fast SQL at scale

Curated tables &
aggregates

e.g. data warehouse

Events / streams

Ordered + durable buffer

Data in motion / event
log

e.g. Kafka-like platform

Discussion: Would you store all four workloads in the same system? Why or why not?

Your Turn: Storage for NYC Taxi

Choose storage based on requirements — not because a technology is familiar.

A

Historical trip files

1 TB+ of raw monthly files
Keep for years
Rarely queried directly

What should storage optimize for?

B

Curated trip tables

Analysts run SQL daily
Aggregations by day / zone
Reliable, repeatable queries

What should storage optimize for?

C

Live taxi events

New events arrive continuously
Consumers need low latency
Temporary durable event history

What should storage optimize for?

Discuss in pairs:

What requirements matter for A, B and C? Would one storage technology be enough?

Think about: cost • latency • query pattern • volume • durability • scalability

NYC Taxi storage — one possible answer

Different requirements lead to different storage choices.

Scenario	What matters most?	Suitable storage
A Historical files	Low cost, durable storage and capacity. Keep 1 TB+ for years; query rarely.	Object storage Google Cloud Storage / Amazon S3
B Curated tables	Efficient daily SQL and aggregations by day or zone; consistent results.	Analytical warehouse BigQuery / Redshift PostgreSQL suits our smaller lab.
C Live events	Record incoming events quickly. Let programs read them within seconds. Keep events so programs can catch up.	Durable event log Kafka, with a retention period

Replay example: a program stops at 10:05 and restarts at 10:15. It reads the retained events from its saved position to catch up.

One system may work at small scale; these three needs often justify different systems.

Storage and Compute: Tightly Coupled or Separate?

A useful modern architecture concept — without tying it to one product.

Traditional / tightly coupled

DATABASE

Storage + Compute

Scaling one often means scaling the whole system.

Modern / separated

STORAGE



COMPUTE

Scale and pay for them independently when the architecture allows it.

Why it matters:

Large amounts of data can remain in inexpensive storage while compute is activated only when processing or querying is needed.

Key Engineering Considerations for Storage

What to ask when evaluating storage

01 Read/write performance

Will it meet the service-level agreement (SLA) and avoid bottlenecks?

02 Scalability

Can it handle future data volume and velocity?

03 Metadata

Are metadata captured? Are data lineage, schema evolution and discovery supported?

Schema evolution & discovery

SCHEMA EVOLUTION

The ability to handle changes in data structure over time. Support means tracking and managing these changes without breaking downstream jobs or requiring manual intervention.

DATA DISCOVERY

How easily engineers, analysts and data scientists find, understand and use datasets—often through a data catalog or metadata layer.

Service-Level Agreement (SLA)

A formal, pre-agreed commitment defining the service a provider will deliver. In a data context, it specifies performance metrics such as:

Latency	How quickly a query or request is answered.
Throughput	How much data is processed or transferred in a given period.
Uptime	How consistently the system is available.

Evaluating Storage: NYC Taxi

Storage is not only about keeping data — it must satisfy engineering and consumer requirements.

1 Performance & SLA

NYC Taxi example

Daily trip dataset must be ready for analysts by 08:00.

Engineering question

Can ingestion, transformation and queries finish on time?

Latency • Throughput • Availability

2 Scalability

NYC Taxi example

Years of taxi trips accumulate and new files keep arriving.

Engineering question

What happens if data volume or arrival rate grows 10×?

Volume • Velocity • Cost

3 Schema Evolution

NYC Taxi example

TLC adds a new field such as `congestion_fee`.

Engineering question

Can old and new files coexist without breaking downstream jobs?

Structure changes over time

4 Metadata & Discovery

NYC Taxi example

An analyst finds a field called `fare_amount`.

Engineering question

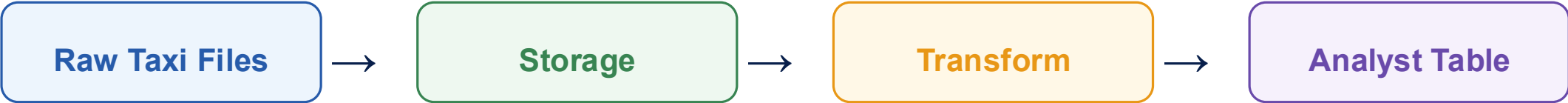
What does it mean? Where did it come from? When was it updated?

Meaning • Discovery • Lineage • Freshness

Takeaway: The right storage choice depends on the service the data must provide — not only on where bytes can be stored.

Your Turn: Is the NYC Taxi Storage Reliable?

A system can be technically “working” and still fail its users.



08:30	The daily table is complete — but the SLA says it must be available by 08:00.	Did the pipeline succeed?
10×	Next year the dataset is 10× larger and queries now take hours.	What should have been considered?
+ column	A new congestion_fee column appears in this month's files.	What happens to old and new data?
fare_amount	A new analyst sees this field but doesn't know its definition or origin.	What information is missing?

Discuss: For each scenario, name the engineering concept involved and one design decision that could reduce the risk.

HSLU

Reveal: What Does “Reliable Storage” Mean?

The four NYC Taxi scenarios map directly to core engineering concepts.

SLA	Dataset ready by 08:00	A technically completed pipeline can still fail the service commitment.	Latency • Throughput • Availability
Scalability	10× more trips	Storage and query design must handle future volume/velocity at acceptable cost.	Scale • Partitioning • Cost
Schema evolution	+ congestion_fee	The platform must manage structural changes without unexpectedly breaking downstream jobs.	Compatibility • Validation
Metadata / discovery	What is fare_amount?	Users need to find, understand and trust data; lineage explains where it came from.	Catalog • Lineage • Freshness

Reliable storage = data is available, scalable, understandable and resilient to change.

Data Temperatures & Storage Selection

Understanding data temperatures



Hot data

Frequently accessed
Requires low-latency storage

e.g., many times per day



Warm data

Accessed occasionally

Weekly / monthly



Cold data

Rarely accessed
Good for archival tiers

e.g., Amazon S3 Glacier

Storage selection depends on

Use cases • Data volume • Access patterns • Ingestion frequency

No one-size-fits-all; each system has trade-offs.